

PROJECTO AW · 2.ª PARCELAR

Foco: primeira entrega da NzolaNet (4 semanas)

Por Onde Começar — Guia Rápido (1.º dia)

Gestão de Utilizadores · Publicações · Comentários

Antes de mergulhar no plano, segue estes passos consoante a tua função:

👉 Para o BE-Dev (Willfredy)

Foco em 3 domínios

4 Semanas

Setup Completo

1. Lê este plano de **fio a pavio** (~20 min).
2. Estuda 1–2 horas a [Secção 9 — O Que Estudar para Backend](#).
3. Faz o vídeo de Nick Chapsas "ASP.NET Core Web API 8 in 1 hour" (YouTube grátis).
4. Cria a solução: `dotnet new sln -n NzolaNet`. Segue os comandos da [Secção 5.12 do plano completo](#).
5. Estabelece o **contrato API** (Secção 6) **antes** de começar a codar — partilha com os FE-Devs.

👉 Para os FE-Devs (Emer, Jeovani, Manuel)

FEITO POR

Willfredy Vieira Dias

1. Lê este plano de fio a pavio (~20 min).
2. Estuda 1–2 horas a [Secção 10 — O Que Estudar para Frontend](#).
3. Vê o vídeo de Mosh Hamedani "Angular Tutorial for Beginners" (YouTube grátis).
4. Cria o projeto: `ng new nzolanet-frontend --style=scss --routing=true`. Setup do Tailwind ou Bootstrap.
5. **Combinem entre vocês** quem fica com qual módulo (ver Secção 3 para a divisão sugerida):
 - **Emer** → `auth/` + perfil
 - **Jeovani** → `feed/` + publicações + comentários
 - **Manuel** → perfis + seguidores + comentários

👉 Para o grupo todo (primeira reunião)

1. **Criar o repositório GitHub** chamado `nzolanet` (ver [Secção 8](#)).
2. **Convidar todos os membros** como colaboradores.
3. **Combinar horário de reuniões** semanais (sugestão: 1h por semana para sincronizar progresso).
4. **Definir o contrato API** em conjunto (Secção 6) — é o documento mais crítico, porque qualquer mudança aqui afeta toda a gente.
5. **Aceitar este plano como fonte da verdade** — qualquer alteração deve ser comunicada ao grupo.

💡 **Conselho:** percam 1–2 dias com os fundamentos antes de codar. Vão ganhar 2 semanas depois.

Índice

- [1. O Que Tem de Funcionar na 2.ª Parcelar](#)
 - [1.5. Fluxo End-to-End \(exemplo: criar publicação\)](#)
 - [2. Arquitetura: Camadas vs MVC + Estrutura do Repositório](#)
 - [3. Divisão da Equipa nesta Fase](#)
 - [4. FRONTEND — O Que Construir](#)
 - [5. BACKEND — O Que Construir](#)
 - [6. Contrato API Mínimo \(2.ª Parcelar\)](#)
 - [7. Cronograma de 4 Semanas](#)
 - [8. Git e GitHub — Resumo Prático](#)
 - [9. O Que Estudar e Dominar — Backend](#)
 - [10. O Que Estudar e Dominar — Frontend](#)
 - [11. Checklist 20/20 da 2.ª Parcelar](#)
 - [12. Próximos Passos Após a Parcelar](#)
 - [13. Glossário de Termos Técnicos](#)
-

1. O Que Tem de Funcionar na 2.ª Parcelar

O enunciado é claro: na 2.ª Parcelar têm de estar **completos e funcionais** três domínios.

1.1. Gestão de Utilizadores

- Registo de novos utilizadores
- Login (com geração de token JWT)
- Recuperação da senha de acesso
- Visualização do perfil próprio e de outros utilizadores
- Edição do perfil (nome, bio, privacidade)
- Alteração da foto de perfil (upload)
- Seguir e deixar de seguir um utilizador

1.2. Gestão de Publicações

- Criar publicações (texto obrigatório; imagem e vídeo opcionais)
- Editar publicações próprias
- Excluir publicações próprias
- Upload de imagens e vídeos
- Visualizar publicações em **ordem cronológica**
- Visualização de conteúdo multimédia (imagens e player de vídeo)

- Lista das publicações de um utilizador (no perfil dele)

1.3. Gestão de Comentários

- Adicionar comentários a uma publicação
- Editar comentários próprios
- Excluir comentários próprios
- Visualizar a lista de comentários de uma publicação

1.4. O Que **Não Entra** na 2.^a Parcelar (mas a infraestrutura já pode existir)

Funcionalidade	Status na 2. ^a Parcelar
Bazes (likes)	Adiada para Exame Final
Feed personalizado de seguidos	Adiada (mostrar feed cronológico simples basta)
Notificações	Adiada
Perfis privados (visibilidade condicional)	Adiada (assumir tudo público nesta fase)
Moderação por administrador	Adiada

💡 **Decisão técnica:** podes já criar as **tabelas** **Bazes** , **Notifications** , **Follows** na BD desde o início (já estão no schema). Apenas não implementas os endpoints/UI até depois da parcelar. Isto evita migrações dolorosas na fase final.

1.5. Fluxo End-to-End (exemplo: "criar publicação")

Para perceber como tudo se interliga, segue o caminho de um clique até à base de dados e de volta. **Estuda este diagrama** — é o "santo graal" da compreensão do sistema.

```
1. UTILIZADOR clica em "Publicar" no Angular
↓
2. ANGULAR (CreatePostComponent)
  - Reactive Form recolhe { content, image, video }
  - PostService monta FormData (multipart)
  - HttpClient envia: POST /api/posts
↓
3. JwtInterceptor (Angular)
  - Lê o token do localStorage
  - Adiciona header: Authorization: Bearer <jwt>
↓ — HTTP via internet —
4. ASP.NET Web API recebe o pedido
  - Middleware de autenticação valida o JWT
  - Identifica o utilizador pelos claims do token
↓
5. PostsController.Create(CreatePostDto dto)
  - [Authorize] confirma que está autenticado
  - Recebe o DTO já deserializado
  - Chama PostService.CreateAsync(dto, userId)
↓
6. PostService (camada de regras de negócio)
  - Valida o conteúdo
  - Se há imagem/vídeo: chama StorageService.SaveAsync
    → grava ficheiro em wwwroot/uploads/...
    → devolve URL "/uploads/media/abc.jpg"
  - Cria entidade Post { UserId, Content, ImageUrl, VideoUrl }
  - Chama PostRepository.AddAsync(post)
↓
7. PostRepository (camada de acesso a dados)
  - dbContext.Posts.Add(post)
  - await dbContext.SaveChangesAsync()
↓
8. ENTITY FRAMEWORK CORE traduz em SQL:
  INSERT INTO Posts (UserId, Content, ImageUrl, ...) VALUES (...)
↓
9. SQL SERVER grava na tabela Posts
  - retorna o Id auto-gerado
↓ — caminho de volta —
10. PostService mapeia Post → PostDto via AutoMapper
11. PostsController devolve 201 Created + PostDto (JSON)
12. ANGULAR recebe o PostDto
  - PostService emite o novo post via Observable
  - FeedComponent atualiza a lista (prepend)
  - Toast: "Publicação criada com sucesso!"
13. UTILIZADOR vê a publicação aparecer no topo do feed
```

Mensagem-chave: cada camada tem **uma responsabilidade clara**. Não há lógica de BD no Controller. Não há HTTP na Service. Isto é "Separação de Camadas" — exatamente o que o enunciado pede.

2. Arquitetura: Camadas vs MVC + Estrutura do Repositório

2.0. O enunciado pede MVC? Não. Qual é a melhor estrutura?

Pergunta crítica respondida: o enunciado **não exige MVC** (Model-View-Controller). O que está escrito, na secção *Requisitos Técnicos*, é: "Deve ser utilizada uma **arquitetura de separação de camadas** (Ex: Repositórios, Serviços, Controllers etc.)" e "Deve ser utilizado **DTOs**".

Porque é que MVC clássico NÃO é a estrutura certa aqui:

- O **MVC clássico** assume que o servidor também gera as **Views** (HTML renderizado no backend, como ASP.NET MVC com Razor). No nosso caso, **a View é o Angular** — uma SPA independente que corre no browser.
- O backend é uma **ASP.NET Web API**: só devolve **JSON**, nunca HTML. Logo não há camada "View" no servidor — só existiria a parte "Controller" do MVC, o que torna o rótulo "MVC" enganador e incompleto.
- Forçar MVC obrigaria a misturar responsabilidades (acesso a dados dentro dos controllers, por exemplo) — exatamente o oposto do que o enunciado premeia.

A melhor estrutura (e a que adotamos): Arquitetura em Camadas (N-Tier / Clean-lite).

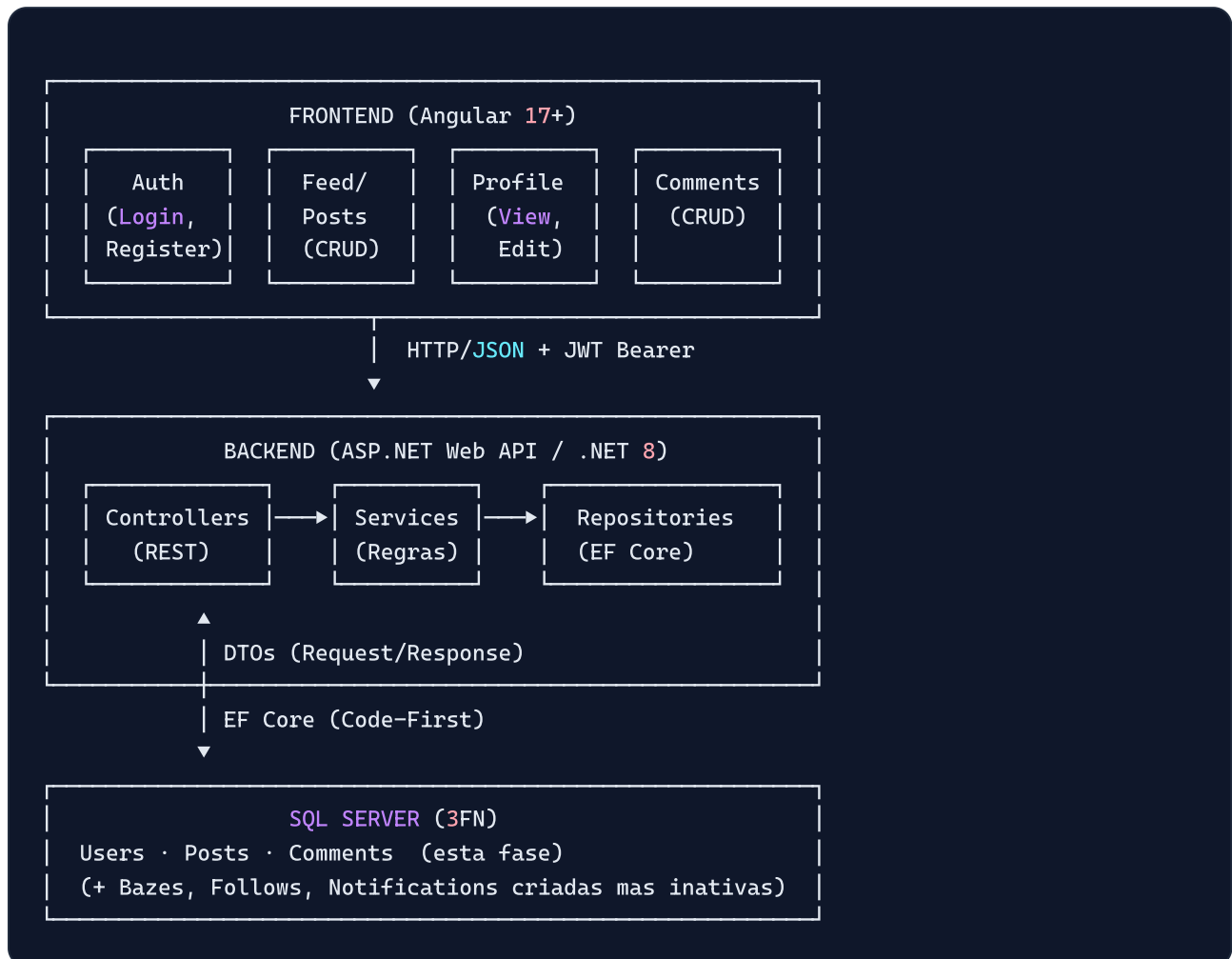
Camada	Projeto .NET	Responsabilidade
Apresentação	<code>NzolaNet.Api</code>	Controllers REST, Middleware, <code>Program.cs</code> . Recebe HTTP, devolve DTOs. <i>Sem lógica de negócio.</i>
Aplicação	<code>NzolaNet.Application</code>	Services (regras de negócio), DTOs, Validators, Mappings, Interfaces. <i>Sem EF Core.</i>
Domínio	<code>NzolaNet.Domain</code>	Entidades puras + interfaces de Repositório. <i>Sem dependências externas.</i>
Infraestrutura	<code>NzolaNet.Infrastructure</code>	EF Core (DbContext, Migrations), Repositórios, JWT, Hash, Upload, Email.

✓ Esta divisão **cumprе literalmente** o enunciado ("Repositórios, Serviços, Controllers") e isola cada responsabilidade. O fluxo é sempre **Controller** → **Service** → **Repository** → **EF Core** → **SQL Server**, com **DTOs** a entrar e sair da API (nunca expomos entidades).

No frontend a estrutura certa não é MVC mas sim a **arquitetura modular do Angular**: `core/` (singletons, guards, interceptors), `shared/` (componentes reutilizáveis), `layouts/` e `features/` (módulos por domínio, lazy-loaded).

✦ **Conclusão:** *MVC não é pedido nem é o ideal.* A combinação **Camadas (backend) + Arquitetura Modular Angular (frontend) + DTOs** é a leitura mais fiel do enunciado e a mais segura para 20/20.

2.1. Diagrama de Camadas



2.2. Estrutura real do repositório (já criada — esqueleto vazio)

O esqueleto de pastas/ficheiros **já existe no repositório** (ficheiros vazios, prontos a preencher). O âmbito está limitado **apenas à 2.ª Parcelar** (Utilizadores, Publicações, Comentários + Autenticação e Seguir). Bazes, Notificações e Feed personalizado **não** foram criados nesta fase.

```

NzolaNet/
├── index.html           ← Landing page (GitHub Pages) com acesso aos planos
├── README.md
├── .gitignore
├── backend/             ← ASP.NET Web API (Willfredy) – arquitetura em camadas
│   ├── NzolaNet.sln
│   ├── NzolaNet.Api/    (Controllers, Middleware, Program.cs, appsettings)
│   ├── NzolaNet.Application/ (Services, DTOs, Validators, Mappings, Interfaces, Except
│   ├── NzolaNet.Domain/   (Entities, Interfaces/Repositories)
│   └── NzolaNet.Infrastructure/ (Data/EF Core, Repositories, Services: JWT/Hash/Email/Sto
├── frontend/           ← Angular (3 colegas) – arquitetura modular
│   ├── proxy.conf.json
│   └── src/ (styles, environments, app/{core,shared,layouts,features})
├── docs/               ← API_CONTRACT.md, database/ (schema + ERD), relatorio/
└── extras/             ← Documentação do projeto (Enunciado, Planos, build-pdf)

```

3. Divisão da Equipa nesta Fase

Elemento	Branch Git	Tarefas na 2.ª Parcelar
Emer Tavares (FE-Dev 1)	<code>frontend/feature/auth</code>	Módulo <code>auth/</code> (login, register, recover) + <code>JwtInterceptor</code> + <code>AuthGuard</code> + <code>edit-profile/</code> + <code>AuthService</code>
Jeovani Sassombo (FE-Dev 2)	<code>frontend/feature/feed-posts</code>	<code>feed-page/</code> (cronológico simples), <code>create-post/</code> , <code>edit-post/</code> , <code>post-card</code> (shared), <code>PostService</code> , <code>UploadService</code> , <code>media-preview</code> (shared)
Manuel Sulo (FE-Dev 3)	<code>frontend/feature/profile-comments</code>	<code>profile-page/</code> , <code>followers-list/</code> , <code>comment-list/</code> , <code>comment-form/</code> , <code>comment-item/</code> , <code>UserService</code> (follow/unfollow), <code>CommentService</code>
BE-Dev (Willfredy)	<code>backend/feature/*</code>	Toda a API: Auth + Users + Posts + Comments + Follow, JWT, upload de ficheiros, regras de negócio

Componentes shared: `navbar`, `sidebar`, `user-avatar`, `post-card`, `comment-item`, `loading-spinner`, `confirm-dialog`, `media-preview`. Quem cria é "dono"; alterações por outros via PR.

4. FRONTEND — O Que Construir

4.1. Tecnologias mínimas para a 2.ª Parcelar

Tecnologia	Papel	Indispensável?
Angular 17+	Framework	✓ Sim
TypeScript 5+	Linguagem	✓ Sim
Angular Router (com lazy loading)	Navegação	✓ Sim
Angular Reactive Forms	Formulários	✓ Sim
Angular HttpClient	Comunicação REST	✓ Sim
Angular Guards + Interceptors	Proteção rotas + JWT	✓ Sim
RxJS (Observable, BehaviorSubject)	Reatividade	✓ Sim
TailwindCSS ou Bootstrap 5	Estilização responsiva	✓ Sim (1 dos 2)
<code>ngx-toastr</code>	Toasts de feedback	● Recomendado
<code>date-fns</code>	Formatação "há 2 horas"	● Recomendado
<code>jwt-decode</code>	Decodificar payload do token	● Recomendado

4.2. Estrutura mínima de pastas (Frontend — 2.ª Parcelar)

Nota de implementação: usamos **Standalone Components (Angular 17+)** com **rotas por feature** (`*.routes.ts` + `loadChildren` / `loadComponent`) em vez de `NgModule` . É a abordagem moderna e mais simples. Cada componente tem `.ts` , `.html` e `.scss` . Os **comentários** não têm feature próprio: vivem como componentes partilhados (`comment-item` , `comment-form`) usados no `post-detail` .

```

frontend/
├── proxy.conf.json           # /api → http://localhost:5000
├── src/
│   ├── styles.scss
│   ├── environments/
│   │   ├── environment.ts    # apiUrl: 'http://localhost:5000/api'
│   │   └── environment.prod.ts
│   └── app/
│       ├── app.component.{ts,html,scss}
│       ├── app.config.ts     # providers (HttpClient, interceptors, router)
│       ├── app.routes.ts     # rotas raiz + lazy loading dos features
│       ├── core/
│       │   ├── guards/       # auth.guard.ts, guest.guard.ts
│       │   ├── interceptors/ # jwt.interceptor.ts, error.interceptor.ts
│       │   ├── models/       # user, post, comment, auth, paged-result (.model.ts)
│       │   └── services/     # auth, user, post, comment, upload (.service.ts)
│       ├── shared/
│       │   ├── components/   # navbar, post-card, comment-item, comment-form,
│       │   │               # user-avatar, media-preview, loading-spinner, conf
│       │   └── pipes/        # time-ago.pipe.ts
│       ├── layouts/          # public-layout, private-layout
│       └── features/
│           ├── auth/         # login, register, forgot-password, reset-password +
│           ├── feed/         # feed-page, create-post + feed.routes.ts
│           ├── posts/        # post-detail, edit-post + posts.routes.ts
│           └── profile/      # profile-page, edit-profile, followers-list + profil

```

⚙️ **Como inicializar:** corre `ng new` num diretório temporário e copia os ficheiros gerados (`angular.json` , `package.json` , `tsconfig*.json` , `main.ts` , `index.html`) para `frontend/` , mantendo a árvore `src/app/` já criada. Ou gera os componentes com `ng generate component features/auth/login --standalone` apontando para estas pastas.

4.3. Ecrãs mínimos (10 ecrãs) que têm de existir

1. **Login** — formulário (email + password)
2. **Registo** — formulário (username, email, password, fullName)
3. **Recuperar senha** — formulário de pedido (email)
4. **Reset senha** — formulário com token + nova senha
5. **Feed principal** — lista cronológica de publicações + botão "criar"
6. **Criar publicação** — modal/página com textarea + upload de imagem/vídeo
7. **Detalhe da publicação** — publicação + lista de comentários + form de adicionar comentário
8. **Editar publicação** — formulário pré-preenchido
9. **Perfil do utilizador** — info do user + lista das suas publicações + botão Seguir/Deixar de seguir
10. **Editar perfil** — formulário (nome, bio, privacidade, foto)

4.4. Modelos TypeScript (espelham os DTOs do backend)

TYPESCRIPT

```
// core/models/user.model.ts
export interface User {
  id: number;
  username: string;
  email: string;
  fullName: string;
  bio?: string;
  profilePhotoUrl?: string;
  isPrivate: boolean;
  isAdmin: boolean;
}

export interface UserProfile extends User {
  followersCount: number;
  followingCount: number;
  postsCount: number;
  isFollowing: boolean;
}

// core/models/post.model.ts
export interface Post {
  id: number;
  authorId: number;
  authorName: string;
  authorUsername: string;
  authorPhotoUrl?: string;
  content: string;
  imageUrl?: string;
  videoUrl?: string;
  commentsCount: number;
  createdAt: string;
  updatedAt: string;
}

// core/models/comment.model.ts
export interface Comment {
  id: number;
  postId: number;
  authorId: number;
  authorName: string;
  authorPhotoUrl?: string;
  content: string;
  isEditable: boolean;
  createdAt: string;
}

// core/models/auth.model.ts
export interface LoginRequest { email: string; password: string; }
export interface RegisterRequest {
  username: string; email: string; password: string; fullName: string;
```

```
}
export interface AuthResponse { token: string; expiration: string; user: User; }
```

⚠️ Repara que **não** existe `bazesCount` nem `userHasBazed` nesta fase — só nesses campos quando avançarmos para o Exame Final.

4.5. Services principais

TYPESCRIPT

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private apiUrl = environment.apiUrl;
  currentUser$ = new BehaviorSubject<User | null>(this.loadUser());

  login(data: LoginRequest): Observable<AuthResponse> {
    return this.http.post<AuthResponse>(`${this.apiUrl}/auth/login`, data)
      .pipe(tap(res => this.saveSession(res)));
  }

  register(data: RegisterRequest): Observable<AuthResponse> { /* ... */ }
  forgotPassword(email: string): Observable<void> { /* ... */ }
  resetPassword(token: string, newPassword: string): Observable<void> { /* ... */ }
  logout(): void { localStorage.clear(); this.currentUser$.next(null); }
  getToken(): string | null { return localStorage.getItem('token'); }
  isAuthenticated(): boolean { return !!this.getToken(); }
}
```

4.6. Interceptor JWT (essencial)

TYPESCRIPT

```
export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).getToken();
  if (token) req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
  return next(req);
};
```

4.7. AuthGuard

TYPESCRIPT

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isAuthenticated()) return true;
  router.navigate(['/auth/login']);
  return false;
};
```

4.8. Regras de UI obrigatórias

- **Botão "Editar/Apagar"** só aparece se o `post.authorId == currentUserId` (idem para comentários).
- **Feed cronológico** (`createdAt DESC`) — paginação simples ou botão "Carregar mais" basta.
- **Upload com pré-visualização**: mostrar a imagem/vídeo antes do submit.
- **Responsividade**: testar em 320 px (mobile), 768 px (tablet), 1024 px (desktop).
- **Feedback visual**: spinners durante chamadas HTTP, toasts em todos os sucessos/erros.
- **Validações client-side**: email válido, password mínimo 8 caracteres, campos obrigatórios.

5. BACKEND — O Que Construir

5.1. Pacotes NuGet mínimos (2.ª Parcelar)

Pacote	Indispensável?
<code>Microsoft.EntityFrameworkCore.SqlServer</code>	✓ Sim
<code>Microsoft.EntityFrameworkCore.Tools</code>	✓ Sim (migrations)
<code>Microsoft.AspNetCore.Authentication.JwtBearer</code>	✓ Sim
<code>System.IdentityModel.Tokens.Jwt</code>	✓ Sim
<code>BCrypt.Net-Next</code>	✓ Sim (hash senha)
<code>AutoMapper.Extensions.Microsoft.DependencyInjection</code>	● Recomendado
<code>FluentValidation.AspNetCore</code>	● Recomendado
<code>Swashbuckle.AspNetCore</code>	✓ Sim (Swagger)

5.2. Estrutura da solução

```
backend/
├── NzolaNet.sln
├── NzolaNet.Api/                                ← Camada de Apresentação
│   ├── Program.cs · appsettings.json · Properties/launchSettings.json
│   ├── Controllers/                            # AuthController, UsersController, PostsController, Commen
│   ├── Middleware/                             # ExceptionHandlingMiddleware
│   └── wwwroot/                                # uploads servidos estaticamente (gitignored)
├── NzolaNet.Application/                       ← Camada de Aplicação (regras de negócio)
│   ├── Interfaces/                            # IAuthService, IUserService, IPostService, ICommentService
│   │   ├── IFollowService, IStorageService, IJwtTokenService,
│   │   └── IPasswordHasher, IEmailService
│   ├── Services/                              # AuthService, UserService, PostService, CommentService, F
│   ├── DTOs/                                  # Auth/ · Users/ · Posts/ · Comments/ · Common/ (PagedResu
│   ├── Validators/                            # RegisterDto, CreatePostDto, CreateCommentDto (FluentValid
│   ├── Mappings/                              # AutoMapperProfile
│   └── Exceptions/                            # NotFound, Forbidden, BadRequest, Conflict
├── NzolaNet.Domain/                           ← Camada de Domínio (pura)
│   ├── Entities/                              # User, Post, Comment, Follow, PasswordResetToken
│   └── Interfaces/Repositories/               # IUserRepository, IPostRepository, ICommentRepository,
│   │   └── IFollowRepository, IPasswordResetTokenRepository
└── NzolaNet.Infrastructure/                   ← Camada de Infraestrutura
    ├── Data/
    │   ├── ApplicationDbContext.cs
    │   ├── Configurations/                    # Fluent API por entidade
    │   ├── Migrations/                        # geradas pelo EF Core
    │   └── Seed/                              # DbSeeder
    ├── Repositories/                          # implementações EF Core dos repositórios
    └── Services/                              # JwtTokenService, PasswordHasher, EmailService, LocalFileS
```

 **Dependências entre projetos:** `Api` → `Application` → `Domain` e `Infrastructure` → `Application` + `Domain`. O `Domain` **não depende de ninguém**. As interfaces de serviços de infraestrutura (JWT, Hash, Email, Storage) ficam em `Application/Interfaces` e são implementadas em `Infrastructure/Services` — assim a Aplicação depende de **abstrações**, não de detalhes técnicos.

5.3. Base de Dados — Tabelas mínimas (2.^a Parcelar)

Foco em **Users, Posts, Comments, Follows, PasswordResetTokens**. As outras (`Bazes`, `Notifications`) podem ser criadas desde já para evitar migrações futuras dolorosas, mas **sem endpoints/serviços** nesta fase.

```

CREATE DATABASE NzolaNetDB;
GO
USE NzolaNetDB;
GO

CREATE TABLE Users (
    Id                INT IDENTITY(1,1) PRIMARY KEY,
    Username          NVARCHAR(50)  NOT NULL UNIQUE,
    Email             NVARCHAR(100) NOT NULL UNIQUE,
    PasswordHash      NVARCHAR(255) NOT NULL,
    FullName          NVARCHAR(100) NOT NULL,
    Bio               NVARCHAR(500) NULL,
    ProfilePhotoUrl   NVARCHAR(500) NULL,
    IsPrivate         BIT NOT NULL DEFAULT 0,
    IsAdmin           BIT NOT NULL DEFAULT 0,
    CreatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    UpdatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE()
);
CREATE INDEX IX_Users_Username ON Users(Username);
CREATE INDEX IX_Users_Email    ON Users(Email);

CREATE TABLE Posts (
    Id                INT IDENTITY(1,1) PRIMARY KEY,
    UserId            INT NOT NULL,
    Content           NVARCHAR(5000) NOT NULL,
    ImageUrl          NVARCHAR(500) NULL,
    VideoUrl          NVARCHAR(500) NULL,
    CreatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    UpdatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    IsDeleted         BIT NOT NULL DEFAULT 0,
    CONSTRAINT FK_Posts_Users FOREIGN KEY (UserId) REFERENCES Users(Id) ON DELETE CASCADE
);
CREATE INDEX IX_Posts_UserId_CreatedAt ON Posts(UserId, CreatedAt DESC);
CREATE INDEX IX_Posts_CreatedAt ON Posts(CreatedAt DESC);

CREATE TABLE Comments (
    Id                INT IDENTITY(1,1) PRIMARY KEY,
    PostId            INT NOT NULL,
    UserId            INT NOT NULL,
    Content           NVARCHAR(1000) NOT NULL,
    CreatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    UpdatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    IsDeleted         BIT NOT NULL DEFAULT 0,
    CONSTRAINT FK_Comments_Posts FOREIGN KEY (PostId) REFERENCES Posts(Id) ON DELETE CASCADE,
    CONSTRAINT FK_Comments_Users FOREIGN KEY (UserId) REFERENCES Users(Id) ON DELETE NO ACTION
);
CREATE INDEX IX_Comments_PostId_CreatedAt ON Comments(PostId, CreatedAt);

CREATE TABLE Follows (
    Id                INT IDENTITY(1,1) PRIMARY KEY,
    FollowerId        INT NOT NULL,
    FollowedId        INT NOT NULL,
    CreatedAt         DATETIME2 NOT NULL DEFAULT GETUTCDATE(),

```

```

CONSTRAINT FK_Follows_Follower FOREIGN KEY (FollowerId) REFERENCES Users(Id) ON DELETE
CONSTRAINT FK_Follows_Followed FOREIGN KEY (FollowedId) REFERENCES Users(Id) ON DELETE
CONSTRAINT UQ_Follows_Pair UNIQUE (FollowerId, FollowedId),
CONSTRAINT CK_Follows_NotSelf CHECK (FollowerId <> FollowedId)
);

CREATE TABLE PasswordResetTokens (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    UserId INT NOT NULL,
    TokenHash NVARCHAR(255) NOT NULL UNIQUE,
    ExpiresAt DATETIME2 NOT NULL,
    UsedAt DATETIME2 NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT GETUTCDATE(),
    CONSTRAINT FK_PRT_Users FOREIGN KEY (UserId) REFERENCES Users(Id) ON DELETE CASCADE
);

```

5.4. Endpoints mínimos (REST)

AuthController — /api/auth

POST	/auth/register	body: RegisterDto	→ 201 AuthResponseDto
POST	/auth/login	body: LoginDto	→ 200 AuthResponseDto
POST	/auth/forgot-password	body: ForgotPasswordDto	→ 202 MessageDto
POST	/auth/reset-password	body: ResetPasswordDto	→ 200 MessageDto

UsersController — /api/users

GET	/users/{id}	[Authorize]	→ UserProfileDto
PUT	/users/me	[Authorize]	→ UserProfileDto
PUT	/users/me/photo	[Authorize] (multipart)	→ UserProfileDto
POST	/users/{id}/follow	[Authorize]	→ MessageDto
DELETE	/users/{id}/follow	[Authorize]	→ MessageDto
GET	/users/{id}/followers	[Authorize]	→ PagedResultDto<UserDto>
GET	/users/{id}/following	[Authorize]	→ PagedResultDto<UserDto>

PostsController — /api/posts

POST	/posts	[Authorize] (multipart)	→ PostDto
GET	/posts/{id}	[Authorize]	→ PostDto
PUT	/posts/{id}	[Authorize] (autor)	→ PostDto
DELETE	/posts/{id}	[Authorize] (autor)	→ MessageDto
GET	/posts/user/{userId}	[Authorize]	→ PagedResultDto<PostDto>
GET	/posts?page=&pageSize=	[Authorize]	→ PagedResultDto<PostDto> # feed

CommentsController

GET	/posts/{postId}/comments	[Authorize]	→ PagedResultDto<CommentDto>
POST	/posts/{postId}/comments	[Authorize]	→ CommentDto
PUT	/comments/{id}	[Authorize] (autor)	→ CommentDto
DELETE	/comments/{id}	[Authorize] (autor)	→ MessageDto

💡 Para a 2.^a Parcelar, o **feed simples** é `GET /posts?page=&pageSize=` ordenado por `CreatedAt DESC` (todas as publicações públicas). No Exame Final criamos o `FeedController` personalizado de seguidos.

5.5. DTOs essenciais

CSHARP

```
// Auth
public record RegisterDto(string Username, string Email, string Password, string FullName);
public record LoginDto(string Email, string Password);
public record AuthResponseDto(string Token, DateTime Expiration, UserDto User);

// Users
public record UserDto(int Id, string Username, string Email, string FullName,
    string? Bio, string? ProfilePhotoUrl, bool IsPrivate, bool IsAdmin);

public record UserProfileDto(int Id, string Username, string FullName, string? Bio,
    string? ProfilePhotoUrl, bool IsPrivate,
    int FollowersCount, int FollowingCount, int PostsCount, bool IsFollowing);

public record UpdateProfileDto(string FullName, string? Bio, bool IsPrivate);

// Posts
public record PostDto(int Id, int AuthorId, string AuthorName, string AuthorUsername,
    string? AuthorPhotoUrl, string Content, string? ImageUrl, string? VideoUrl,
    int CommentsCount, DateTime CreatedAt, DateTime UpdatedAt);

public record CreatePostDto(string Content, IFormFile? Image, IFormFile? Video);
public record UpdatePostDto(string Content, IFormFile? Image, IFormFile? Video);

// Comments
public record CommentDto(int Id, int PostId, int AuthorId, string AuthorName,
    string? AuthorPhotoUrl, string Content, bool IsEditable, DateTime CreatedAt);

public record CreateCommentDto(string Content);
public record UpdateCommentDto(string Content);

// Shared
public record PagedResultDto<T>(IEnumerable<T> Items, int Page, int PageSize,
    int TotalCount, int TotalPages);
public record MessageDto(string Message);
```

5.6. Regras de negócio críticas (Services)

CSHARP

```
// PostService.cs - Edição apenas pelo autor
public async Task<PostDto> UpdateAsync(int postId, UpdatePostDto dto, int currentUserId)
{
    var post = await _postRepo.GetByIdAsync(postId)
        ?? throw new NotFoundException("Publicação não encontrada.");

    if (post.UserId != currentUserId)
        throw new ForbiddenException("Apenas o autor pode editar esta publicação.");

    post.Content = dto.Content;
    post.UpdatedAt = DateTime.UtcNow;

    if (dto.Image is not null) post.ImageUrl = await _storage.SaveAsync(dto.Image, "media")
    if (dto.Video is not null) post.VideoUrl = await _storage.SaveAsync(dto.Video, "media")

    await _postRepo.UpdateAsync(post);
    return _mapper.Map<PostDto>(post);
}
```

CSHARP

```
// CommentService.cs - Edição apenas pelo autor
public async Task<CommentDto> UpdateAsync(int commentId, UpdateCommentDto dto, int currentU
{
    var comment = await _commentRepo.GetByIdAsync(commentId)
        ?? throw new NotFoundException("Comentário não encontrado.");

    if (comment.UserId != currentUserId)
        throw new ForbiddenException("Apenas o autor pode editar este comentário.");

    comment.Content = dto.Content;
    comment.UpdatedAt = DateTime.UtcNow;
    await _commentRepo.UpdateAsync(comment);
    return _mapper.Map<CommentDto>(comment);
}
```

```
// FollowService.cs - Seguir
public async Task FollowAsync(int targetUserId, int currentUserId)
{
    if (targetUserId == currentUserId)
        throw new BadRequestException("Não podes seguir-te a ti próprio.");

    if (await _followRepo.ExistsAsync(currentUserId, targetUserId))
        throw new ConflictException("Já segues este utilizador.");

    await _followRepo.AddAsync(new Follow {
        FollowerId = currentUserId, FollowedId = targetUserId
    });
}
```

5.7. Upload de ficheiros

```
public class LocalFileStorageService : IStorageService
{
    private static readonly string[] _imgExt = [".jpg", ".jpeg", ".png", ".webp"];
    private static readonly string[] _vidExt = [".mp4", ".webm", ".mov"];
    private const long MaxImageSize = 10 * 1024 * 1024;
    private const long MaxVideoSize = 50 * 1024 * 1024;

    public async Task<string> SaveAsync(IFormFile file, string folder)
    {
        var ext = Path.GetExtension(file.FileName).ToLowerInvariant();
        var isImage = _imgExt.Contains(ext);
        var isVideo = _vidExt.Contains(ext);

        if (!isImage && !isVideo)
            throw new BadRequestException("Formato de ficheiro não suportado.");
        if (isImage && file.Length > MaxImageSize)
            throw new BadRequestException("Imagem demasiado grande (máx. 10 MB).");
        if (isVideo && file.Length > MaxVideoSize)
            throw new BadRequestException("Vídeo demasiado grande (máx. 50 MB).");

        var fileName = $"{Guid.NewGuid()}{ext}";
        var dir = Path.Combine("wwwroot", "uploads", folder);
        Directory.CreateDirectory(dir);

        await using var stream = new FileStream(Path.Combine(dir, fileName), FileMode.Create);
        await file.CopyToAsync(stream);

        return $"/uploads/{folder}/{fileName}";
    }
}
```

5.8. Program.cs (mínimo viável)

CSHARP

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<ApplicationDbContext>(opt =>
    opt.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var jwtKey = builder.Configuration["Jwt:Key"]!;
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opt => {
        opt.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey)),
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            ValidateIssuer = true, ValidateAudience = true, ValidateLifetime = true
        };
    });

builder.Services.AddAuthorization();

builder.Services.AddCors(opt => opt.AddPolicy("Angular", p => p
    .WithOrigins("http://localhost:4200")
    .AllowAnyHeader().AllowAnyMethod()));

// DI - Repositórios
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddScoped<IPostRepository, PostRepository>();
builder.Services.AddScoped<ICommentRepository, CommentRepository>();
builder.Services.AddScoped<IFollowRepository, FollowRepository>();

// DI - Serviços
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IUserService, UserService>();
builder.Services.AddScoped<IPostService, PostService>();
builder.Services.AddScoped<ICommentService, CommentService>();
builder.Services.AddScoped<IFollowService, FollowService>();
builder.Services.AddScoped<IStorageService, LocalFileStorageService>();

builder.Services.AddAutoMapper(typeof(AutoMapperProfile).Assembly);
builder.Services.AddControllers();
builder.Services.AddSwaggerGen();

var app = builder.Build();
app.UseSwagger(); app.UseSwaggerUI();
app.UseCors("Angular");
app.UseAuthentication(); app.UseAuthorization();
app.UseStaticFiles();
app.MapControllers();
app.Run();
```

6. Contrato API Mínimo (2.ª Parcelar)

Este é um **subconjunto** do `docs/API_CONTRACT.md`. Para a 2.ª Parcelar, basta este contrato funcionar end-to-end.

MARKDOWN

API Contract – NzolaNet (v0.5 – 2.ª Parcelar)

Base URL: `http://localhost:5000/api`

Autenticação: `Authorization: Bearer <JWT>`

Auth

POST	<code>/auth/register</code>	body: <code>RegisterDto</code>	→ 201 <code>AuthResponseDto</code>
POST	<code>/auth/login</code>	body: <code>LoginDto</code>	→ 200 <code>AuthResponseDto</code>
POST	<code>/auth/forgot-password</code>	body: <code>{ email }</code>	→ 202 <code>MessageDto</code>
POST	<code>/auth/reset-password</code>	body: <code>{ token, password }</code>	→ 200 <code>MessageDto</code>

Users

GET	<code>/users/{id}</code>	→ <code>UserProfileDto</code>	
PUT	<code>/users/me</code>	body: <code>UpdateProfileDto</code>	→ <code>UserProfileDto</code>
PUT	<code>/users/me/photo</code>	multipart: <code>photo</code>	→ <code>UserProfileDto</code>
POST	<code>/users/{id}/follow</code>	→ <code>MessageDto</code>	
DELETE	<code>/users/{id}/follow</code>	→ <code>MessageDto</code>	
GET	<code>/users/{id}/followers</code>	→ <code>PagedResultDto<UserDto></code>	
GET	<code>/users/{id}/following</code>	→ <code>PagedResultDto<UserDto></code>	

Posts

POST	<code>/posts</code>	multipart: <code>content, image?, video?</code>	→ 201 <code>PostDto</code>
GET	<code>/posts?page=&pageSize=</code>	→ <code>PagedResultDto<PostDto></code>	# feed cronológico simples
GET	<code>/posts/{id}</code>	→ <code>PostDto</code>	
PUT	<code>/posts/{id}</code>	multipart: <code>content, image?, video?</code>	→ <code>PostDto</code>
DELETE	<code>/posts/{id}</code>	→ <code>MessageDto</code>	
GET	<code>/posts/user/{userId}</code>	→ <code>PagedResultDto<PostDto></code>	

Comments

GET	<code>/posts/{postId}/comments</code>	→ <code>PagedResultDto<CommentDto></code>	
POST	<code>/posts/{postId}/comments</code>	body: <code>{ content }</code>	→ 201 <code>CommentDto</code>
PUT	<code>/comments/{id}</code>	body: <code>{ content }</code>	→ <code>CommentDto</code>
DELETE	<code>/comments/{id}</code>	→ <code>MessageDto</code>	

7. Cronograma de 4 Semanas

Semana	Backend (Willfredy)	Frontend (3 devs)
Semana 1	Setup solução .NET + EF Core + DB schema completo (Migration <code>InitialCreate</code>) + JWT setup + AuthController (register/login)	Setup Angular + Tailwind/Bootstrap + módulo <code>auth/</code> (login, register) + <code>AuthGuard</code> + <code>JwtInterceptor</code> + <code>AuthService</code> + layouts (<code>public-layout</code> , <code>private-layout</code>)
Semana 2	UsersController (perfil, edição, foto, follow/unfollow) + Upload de ficheiros	<code>edit-profile/</code> + <code>profile-page/</code> + componentes <code>shared/</code> (navbar, user-avatar) + <code>UserService</code>
Semana 3	PostsController (CRUD + upload media) + CommentsController (CRUD) + lista cronológica de posts	<code>feed-page/</code> + <code>create-post/</code> + <code>edit-post/</code> + <code>post-card</code> (shared) + <code>comment-list/</code> + <code>comment-form/</code> + <code>PostService</code> + <code>CommentService</code>
Semana 4	Testes manuais via Swagger, correções, polimento, relatório intermédio , Swagger documentado, seed de dados	Responsividade (mobile/tablet/desktop), validações finais, integração ponta-a-ponta com o backend, fix de bugs

Marcos críticos

- **Final da Semana 1:** registo + login a funcionar end-to-end. Token JWT chega ao frontend e é injetado nas chamadas seguintes.
- **Final da Semana 2:** perfil completamente funcional (ver/editar/foto/seguir).
- **Final da Semana 3:** publicações e comentários CRUD completos com upload.
- **Final da Semana 4:** **demo gravada** + relatório parcial pronto.

8. Git e GitHub — Resumo Prático

Para o passo a passo **completo** (criar repo, convidar membros, proteger branches, etc.), vê o [Plano Definitivo Completo](#).

Workflow diário (todos os dias)

BASH

```
# 1. Sincronizar com develop
git checkout develop
git pull origin develop

# 2. Criar/continuar branch da tarefa
git checkout -b frontend/feature/auth      # ou continuar existente
# ... trabalhar ...

# 3. Commitar com convenção
git add .
git commit -m "feat(frontend/auth): implementa login com JWT"
git push -u origin frontend/feature/auth

# 4. Abrir Pull Request no GitHub para `develop`
# 5. Outro membro revê e aprova → squash merge
```

Convenção de commits

```
feat(<escopo>):    nova funcionalidade
fix(<escopo>):     correção de bug
refactor(<escopo>): melhoria sem mudança de comportamento
docs(<escopo>):    alteração de documentação
chore(<escopo>):   config, dependências, manutenção
```

Branches da 2.ª Parcelar

- `main` (protegida — versão estável)
- `develop` (integração contínua)
- `backend/feature/auth`, `backend/feature/users`, `backend/feature/posts`,
`backend/feature/comments`
- `frontend/feature/auth`, `frontend/feature/feed-posts`, `frontend/feature/profile-`
`comments`

Promoção final da parcelar

BASH

```
git checkout main
git pull origin main
git merge develop --no-ff -m "release(v0.5): 2.ª Parcelar - Users, Posts, Comments"
git push origin main
git tag v0.5-parcelar
git push origin --tags
```


9. O Que Estudar e Dominar — Backend

Audiência: Willfredy (BE-Dev). Tópicos ordenados por **prioridade** para a 2.^a Parcelar.

9.1. Fundamentos C# / .NET (Pré-requisito)

Tópico	Por que importa	Recursos sugeridos
Sintaxe C# moderna (records, pattern matching, primary constructors)	Toda a API e DTOs usam isto	Microsoft Learn — C# Tutorial · YouTube: Nick Chapsas ("C# 12 features")
LINQ (Where, Select, OrderBy, Skip/Take, Include)	Todas as queries EF passam por LINQ	YouTube: IAmTimCorey ("LINQ for Beginners") · Codewrinkles — LINQ Crash Course
async/await + Task	Toda a API é assíncrona	Microsoft Learn — Asynchronous programming in C# · YouTube: Raw Coding ("Async Await Explained")
Dependency Injection (DI)	Toda a infraestrutura .NET 8 funciona com DI	YouTube: Nick Chapsas ("Dependency Injection in .NET")

9.2. ASP.NET Core Web API

Tópico	Por que importa	Recursos sugeridos
Criar projeto Web API + Program.cs (minimal hosting)	Setup inicial	YouTube: Nick Chapsas — "ASP.NET Core Web API 8 in 1 hour"
Controllers, ActionResult, Routing (<code>[Route]</code> , <code>[HttpGet]</code> , <code>[HttpPost]</code> , ...)	Toda a API	Patrick God — "Build a REST API with .NET 8" (YouTube, full series)
Model Binding + <code>[FromBody]</code> , <code>[FromForm]</code> , <code>[FromQuery]</code>	Receber dados do frontend	Microsoft Learn — <i>Model Binding in ASP.NET Core</i>
Middleware (Pipeline, custom exception handler)	Tratamento global de erros	YouTube: Milan Jovanović — "Global Exception Handling in ASP.NET"
CORS	Sem CORS o frontend não comunica	YouTube: Nick Chapsas — "CORS in ASP.NET Core"
Swagger / OpenAPI (<code>Swashbuckle.AspNetCore</code>)	Documentar e testar a API	Microsoft Learn — <i>Get started with Swashbuckle</i>

9.3. Entity Framework Core

Tópico	Por que importa	Recursos sugeridos
DbContext, DbSet, OnModelCreating (Fluent API)	Configurar a BD	YouTube: Patrick God — "Entity Framework Core 8 Crash Course"
Code-First Migrations (<code>Add-Migration</code> , <code>Update-Database</code>)	Versionar a BD	IAmTimCorey — "Entity Framework Core 6 Migrations" (YouTube)
Relações 1:N, N:N, Auto-referência (Follows)	Modelar o domínio	Milan Jovanović — "EF Core Relationships"
LINQ to SQL (Where, Include, Select projections, AsNoTracking)	Queries eficientes	Nick Chapsas — "EF Core Performance Tips"
Unique constraints e Check constraints via Fluent API	<code>Bazes</code> e <code>Follows</code>	Microsoft Learn — <i>Indexes (Fluent API)</i>

9.4. SQL Server

Tópico	Por que importa	Recursos sugeridos
T-SQL básico (<code>SELECT</code> , <code>JOIN</code> , <code>WHERE</code> , <code>GROUP BY</code>)	Debug e Seeds	Kudvenkat — "SQL Server Tutorial for Beginners" (YouTube)
Constraints (PK, FK, UNIQUE, CHECK)	Integridade dos dados	Tutorial: W3Schools SQL · SQLBolt
Índices (clustered vs non-clustered)	Performance	YouTube: Brent Ozar — "Index Tuning"
SSMS (SQL Server Management Studio)	Inspecionar a BD manualmente	YouTube: Kudvenkat — "SSMS Tutorial"

9.5. Autenticação e Segurança

Tópico	Por que importa	Recursos sugeridos
JWT (estrutura: header.payload.signature)	Base da autenticação	YouTube: Patrick God — "JWT Authentication in .NET 8"
Hash de password com BCrypt	Nunca guardar passwords em claro	YouTube: Nick Chapsas — "Hashing Passwords in .NET"
Claims e Roles	Identificar utilizador autenticado nos controllers	Microsoft Learn — <i>Claims-based authorization</i>
<code>[Authorize]</code> e Authorization Policies	Proteger endpoints	YouTube: Milan Jovanović — "Authorization in ASP.NET Core"

9.6. Padrões de Arquitetura

Tópico	Por que importa	Recursos sugeridos
Repository Pattern	Isolar acesso a dados	YouTube: Patrick God — "Repository Pattern in .NET"
Service Layer	Concentrar regras de negócio	YouTube: Milan Jovanović — "Service Layer in Clean Architecture"
DTO vs Entity	Não expor entidades à API	YouTube: Nick Chapsas — "DTOs vs Entities"
AutoMapper (configuração de Profiles)	Mapear automaticamente	Documentação oficial: automapper.org · YouTube: Tim Corey
FluentValidation	Validar DTOs de forma elegante	YouTube: Milan Jovanović — "FluentValidation in .NET"

9.7. Upload de Ficheiros

Tópico	Por que importa	Recursos sugeridos
<code>IFormFile</code> , multipart/form-data	Upload de imagens/vídeos	YouTube: Patrick God — "File Upload in ASP.NET Core"
<code>app.UseStaticFiles()</code> + servir <code>wwwroot/uploads</code>	Frontend acede aos uploads	Microsoft Learn — <i>Static files in ASP.NET Core</i>

9.8. Roadmap sugerido (sequência de estudo)

Semana 0 (pré-projeto):

C# moderno + LINQ + async/await
+ Criar primeira Web API "Hello World"

Semana 1:

EF Core (DbContext, Migrations)
JWT Authentication (gerar e validar tokens)
Hash com BCrypt

Semana 2:

Repository + Service Pattern
AutoMapper + DTOs
Upload de ficheiros (IFormFile)

Semana 3:

FluentValidation
Exception Middleware
Swagger documentado com Bearer

Semana 4:

Polish + relatório

9.9. Cursos completos recomendados (escolher 1 ou 2)

- GB **Nick Chapsas — Building a complete REST API in .NET 8** (YouTube grátis)
- GB **Patrick God — .NET 8 Web API & Entity Framework Tutorial (Full Course)** (YouTube grátis)
- BR **Lucas Montano / balta.io / Macoratti.net** (PT-BR)
- GB **Microsoft Learn — Build web APIs with ASP.NET Core** (oficial, grátis)
- GB **Milan Jovanović — Pragmatic Clean Architecture** (curso pago avançado, opcional)

10. O Que Estudar e Dominar — Frontend

Audiência: Emer Tavares, Jeovani Sassombo, Manuel Sulo (FE-Devs). Tópicos ordenados por **prioridade** para a 2.ª Parcelar.

10.1. Fundamentos TypeScript / Angular (Pré-requisito)

Tópico	Por que importa	Recursos sugeridos
TypeScript básico (tipos, interfaces, generics, narrowing)	Toda a app Angular é TypeScript	The Net Ninja — TypeScript Tutorial for Beginners (YouTube) · TypeScript Handbook (oficial)
Arquitetura Angular (Module, Component, Service, Directive, Pipe)	Base de tudo	Angular University — Angular Crash Course · Mosh Hamedani — Angular 17 Tutorial (YouTube)
Standalone Components (Angular 17+)	Mais simples que NgModules	YouTube: Decoded Frontend — "Standalone Components Tutorial"
Component lifecycle (<code>ngOnInit</code> , <code>ngOnDestroy</code>)	Inicialização e limpeza	YouTube: Joshua Morony — "Lifecycle Hooks"
<code>@Input()</code> e <code>@Output()</code>	Comunicação entre componentes	YouTube: Net Ninja — "Angular Tutorial #11: @Input and @Output"

10.2. Roteamento e Navegação

Tópico	Por que importa	Recursos sugeridos
Angular Router (<code><router-outlet></code> , <code>routerLink</code> , <code>Router</code>)	Toda a navegação	YouTube: Decoded Frontend — "Angular Routing Masterclass"
Lazy loading (<code>loadChildren</code>)	Performance e organização	YouTube: Joshua Morony — "Lazy Loading in Angular"
Route Guards (<code>CanActivate</code>)	Proteger rotas autenticadas	YouTube: Code Inspire — "Auth Guard in Angular"
Parâmetros de rota (<code>/profile/:id</code>)	Páginas dinâmicas	YouTube: Mosh Hamedani — "Angular Route Parameters"

10.3. Formulários

Tópico	Por que importa	Recursos sugeridos
Reactive Forms (<code>FormGroup</code> , <code>FormControl</code> , <code>Validators</code>)	Formulários da app (login, register, posts, comments)	YouTube: Angular University — "Reactive Forms in Depth"
Validators customizados	Password forte, confirmar password	YouTube: Decoded Frontend — "Custom Validators"
Submit + tratamento de erros	UX em formulários	YouTube: Joshua Morony — "Form Submission Patterns"

10.4. HTTP e Comunicação com API

Tópico	Por que importa	Recursos sugeridos
<code>HttpClient</code> (<code>get</code> , <code>post</code> , <code>put</code> , <code>delete</code>)	Toda a comunicação REST	YouTube: Net Ninja — "Angular HttpClient"
HttpInterceptor	Injetar JWT automaticamente	YouTube: Joshua Morony — "HTTP Interceptors in Angular"
Upload de ficheiros com <code>FormData</code>	Upload de fotos/vídeos	YouTube: Code Inspire — "File Upload in Angular"

10.5. RxJS (Observable, Pipe Operators)

Tópico	Por que importa	Recursos sugeridos
<code>Observable</code> , <code>subscribe</code> , <code>unsubscribe</code>	Toda a comunicação assíncrona	YouTube: Decoded Frontend — "RxJS Basics"
Operators essenciais: <code>map</code> , <code>filter</code> , <code>switchMap</code> , <code>catchError</code> , <code>tap</code>	Manipulação reativa	YouTube: Joshua Morony — "RxJS Operators You Should Know"
<code>BehaviorSubject</code> (estado partilhado, ex: utilizador autenticado)	Estado da app	YouTube: Decoded Frontend — "BehaviorSubject Explained"
<code>async</code> pipe	Bind automático no template (evita memory leaks)	YouTube: Joshua Morony — "Use async pipe everywhere"

10.6. Estilização e Responsividade

Escolher uma ferramenta: TailwindCSS ou Bootstrap 5. Não misturar.

Tópico	Por que importa	Recursos sugeridos
TailwindCSS (utility-first)	Estilização rápida e moderna	YouTube: Net Ninja — Tailwind CSS Tutorial · Tailwind oficial
Bootstrap 5 (componentes prontos)	Estilização tradicional	YouTube: The Coding Train · Bootstrap oficial
CSS Grid + Flexbox	Base de qualquer layout responsivo	YouTube: Kevin Powell — "Learn CSS Grid + Flexbox"
Mobile-first design + Media queries	Compatibilidade móvel obrigatória	YouTube: Kevin Powell — "Mobile First CSS"

10.7. Padrões e Boas Práticas

Tópico	Por que importa	Recursos sugeridos
Services + Dependency Injection	Lógica fora dos componentes	YouTube: Decoded Frontend — "Angular Services Best Practices"
Componentes reutilizáveis (<code>@Input</code> , <code>post-card</code> , <code>user-@Output</code> , <code>ng-content</code>)	<code>post-card</code> , <code>user-avatar</code> , etc.	YouTube: Joshua Morony — "Reusable Components in Angular"
Smart vs Presentational Components	Arquitetura limpa	Artigo: Dan Wahlin — Angular Architecture Patterns
<code>trackBy</code> em <code>*ngFor</code>	Performance em listas	YouTube: Joshua Morony — "TrackBy in Angular"

10.8. Roadmap sugerido (sequência de estudo)

Semana 0 (pré-projeto):

TypeScript básico
+ Criar primeiro app Angular "Hello World"
+ Estrutura de um component (template + class + style)

Semana 1:

Reactive Forms (login, register)
HttpClient + chamadas básicas
Router + AuthGuard
HttpInterceptor (injeção de JWT)

Semana 2:

Services + BehaviorSubject (utilizador autenticado)
Lazy loading de módulos
TailwindCSS / Bootstrap (responsividade)

Semana 3:

Componentes reutilizáveis (post-card)
Upload de ficheiros com FormData
RxJS operators (switchMap, catchError)
ngx-toastr (feedback ao utilizador)

Semana 4:

Polish responsivo
Acessibilidade básica (alt, aria-label)
Validações finais

10.9. Cursos completos recomendados (escolher 1 ou 2)

- GB **Mosh Hamedani — Angular Tutorial for Beginners** (YouTube, gratuito)
- GB **Net Ninja — Complete Angular Course** (YouTube, gratuito)
- GB **Decoded Frontend — Angular Masterclass** (YouTube, gratuito, avançado)
- GB **Joshua Morony — Modern Angular Tutorials** (YouTube, gratuito, moderno)
- BR **Loiane Groner — Curso Angular** (YouTube PT-BR)
- BR **Felipe Rocha — DevPleno Angular** (YouTube PT-BR)
- GB **Angular University** (cursos pagos, qualidade alta)
- GB **Documentação oficial** — <https://angular.dev> (a melhor referência absoluta)

10.10. Para os 3 FE-Devs: divisão sugerida de estudo

Dev	Tópicos prioritários	Tópicos secundários
Emer (FE-Dev 1 · Auth + Edit Profile)	Reactive Forms · Validators customizados · JWT no LocalStorage · HttpInterceptor · AuthGuard	Upload de foto (FormData)
Jeovani (FE-Dev 2 · Feed + Posts)	HttpClient + paginação simples · Componentes reutilizáveis (<code>post-card</code>) · <code>*ngFor</code> + <code>trackBy</code> · Lazy loading	Upload de imagem/vídeo · <code><video></code> HTML5
Manuel (FE-Dev 3 · Profile + Comments)	RxJS (Observable + async pipe) · Comunicação entre componentes (<code>@Input</code> / <code>@Output</code>) · Route Params (<code>/profile/:id</code>)	Diálogos de confirmação · <code>ng-</code> <code>content</code>

11. Checklist 20/20 da 2.ª Parcelar

Backend ✓

- ☐ Solução .NET com 4 projetos (Api, Application, Domain, Infrastructure)
- ☐ Arquitetura em camadas: **Controller** → **Service** → **Repository**
- ☐ **DTOs em todos os endpoints** (nunca expor entidades)
- ☐ EF Core Code-First com `InitialCreate` migration aplicada
- ☐ **JWT** funcional (register + login)
- ☐ Password com **BCrypt**
- ☐ Recuperação de senha com **token + expiração**
- ☐ `[Authorize]` em todos os endpoints privados
- ☐ **CRUD completo** de Users, Posts, Comments
- ☐ Follow / Unfollow
- ☐ **Upload de imagens e vídeos** com validação de tipo e tamanho
- ☐ **CORS** configurado para `http://localhost:4200`
- ☐ **Swagger** documentado (com Bearer auth)
- ☐ Códigos HTTP corretos (200, 201, 400, 401, 403, 404)
- ☐ Edição/exclusão apenas pelo autor

Frontend ✓

- ☐ Estrutura modular (`core/` , `shared/` , `features/`)
- ☐ **Módulos lazy-loaded** com `loadChildren`
- ☐ **AuthGuard** + **GuestGuard**

- ☐ **JwtInterceptor + ErrorInterceptor**
- ☐ **Modelos TypeScript** espelham os DTOs do backend
- ☐ **Reactive Forms** com validação client-side
- ☐ Feed cronológico funcional (paginação simples)
- ☐ Upload de imagem/vídeo com pré-visualização
- ☐ CRUD de publicações (criar, ver, editar, apagar)
- ☐ CRUD de comentários
- ☐ Perfil próprio e de outros utilizadores
- ☐ Edição de perfil + alteração de foto
- ☐ Follow / Unfollow
- ☐ **Interface responsiva** (mobile, tablet, desktop)
- ☐ Feedback visual (spinners, toasts)
- ☐ Botões Editar/Apagar condicionais

Git e Documentação ✓

- ☐ Repositório monorepo (`backend/` , `frontend/` , `docs/`)
- ☐ Branches protegidas (`main` , `develop`)
- ☐ **PRs com revisão** antes de merge
- ☐ **Convenção de commits** respeitada
- ☐ Tag de versão `v0.5-parcelar`
- ☐ `docs/API_CONTRACT.md` atualizado para v0.5
- ☐ **Relatório parcial** com:
 - Análise dos 3 domínios entregues
 - Capturas de ecrã (login, feed, perfil, comentário)
 - Modelo de dados (ERD parcial)
 - Decisões técnicas tomadas
 - O que ficou para o Exame Final

Demo ✓

- ☐ Gravar vídeo de **3-5 minutos** com:
 1. Registo de utilizador novo
 2. Login
 3. Edição de perfil + upload de foto
 4. Criar publicação com imagem
 5. Editar a publicação
 6. Comentar na publicação
 7. Editar/apagar comentário

8. Seguir outro utilizador

- ☐ Inserir o vídeo na pasta `docs/` do repositório (ou link YouTube unlisted)

12. Próximos Passos Após a Parcelar

Depois de entregar a 2.ª Parcelar, o foco passa para o [Plano Definitivo Completo](#) — secções avançadas que ficam para o **Exame de Época Normal**:

Área	Funcionalidade pós-parcelar
Interações	Bazes (toggle com unicidade)
Feed inteligente	<code>FeedController</code> com publicações de seguidos + paginação
Notificações	Geração automática (baze, comentário, novo seguidor) + UI (sino com badge) + polling
Privacidade	Lógica condicional de perfil privado (acesso só a seguidores)
Moderação	Endpoint admin para remover comentários ofensivos
Polish	Acessibilidade, OnPush change detection, optimistic UI, performance, relatório final

 **Estima-se 4 semanas adicionais** entre a Parcelar e o Exame para implementar tudo isto e entregar com 20/20 final.

Recursos rápidos

-  [Plano Definitivo Completo \(HTML\)](#)
-  [Plano Definitivo Completo \(Markdown\)](#)
-  [Plano Definitivo Completo \(PDF\)](#)
-  [Enunciado original](#)

13. Glossário de Termos Técnicos

Lista de termos usados ao longo do plano. Se vires um termo que não conheces, consulta aqui antes de procurar fora. Inglês porque é convenção universal na indústria — vê notas técnicas no

fim deste glossário.

Geral / Arquitetura

Termo	Significado
API	Application Programming Interface. Conjunto de endpoints que o backend expõe para o frontend consumir.
REST	Representational State Transfer. Estilo de API que usa URLs + verbos HTTP (GET, POST, PUT, DELETE) com JSON.
Endpoint	Um ponto da API. Ex: <code>POST /api/auth/login</code> é um endpoint.
HTTP Method / Verbo	GET (ler), POST (criar), PUT (atualizar), DELETE (apagar), PATCH (atualizar parcial).
JSON	JavaScript Object Notation. Formato de texto para enviar/receber dados.
CORS	Cross-Origin Resource Sharing. Mecanismo que permite o frontend (<code>localhost:4200</code>) chamar o backend (<code>localhost:5000</code>).
Stack	Conjunto de tecnologias do projeto. A nossa stack: Angular + ASP.NET + SQL Server.
Camadas	Divisão do código por responsabilidade. Controller → Service → Repository → BD.

Frontend (Angular)

Termo	Significado
Component	Bloco reutilizável de UI (template HTML + classe TypeScript + estilo SCSS). Ex: <code>PostCardComponent</code> .
Module	Agrupamento de components/services relacionados. Ex: <code>AuthModule</code> .
Service	Classe injetável com lógica reutilizável (não-UI). Ex: <code>AuthService</code> faz login.
Reactive Forms	API do Angular para formulários com validação programática. Alternativa: Template-driven forms.
HttpClient	Cliente HTTP do Angular. Envia pedidos REST ao backend.
Observable (RxJS)	"Promessa" que pode emitir múltiplos valores ao longo do tempo. Usado em <code>HttpClient</code> .
BehaviorSubject	Tipo de Observable que guarda o último valor. Útil para estado partilhado (ex: utilizador autenticado).
Subscribe / Unsubscribe	Começar/parar de "ouvir" um Observable. Quem subscreve tem de unsubscribe (ou usar <code>async</code> pipe).
Guard	Função que decide se uma rota pode ser acedida. Ex: <code>AuthGuard</code> bloqueia se não estiver logado.
Interceptor	"Filtro" que modifica todos os pedidos HTTP. Ex: <code>JwtInterceptor</code> injeta o token.
Lazy Loading	Carregar um módulo só quando o utilizador navega para ele. Melhora performance.
<code>@Input()</code> / <code>@Output()</code>	Decoradores para passar dados (de pai para filho) e emitir eventos (de filho para pai).
<code>*ngFor</code> / <code>*ngIf</code>	Diretivas estruturais. Repetir elementos em lista / mostrar condicionalmente.
Pipe	Transforma um valor no template. Ex: <code>{{ date timeAgo }}</code> → "há 2 horas".
Directive	Lógica anexada a um elemento. Ex: <code>[(ngModel)]</code> é uma directiva.
Standalone Component (Angular 17+)	Component sem precisar de <code>NgModule</code> . Mais simples e moderno.

Termo	Significado
FormData	Objeto JavaScript que serve para enviar ficheiros (multipart) ao backend.
Signal (Angular 17+)	Nova forma reativa de gerir estado, alternativa a BehaviorSubject .
JWT	JSON Web Token. String assinada que prova quem és. Composta por header.payload.signature em Base64.
localStorage	Armazenamento do browser persistente (sobrevive ao refresh). Usamos para guardar o JWT.
Smooth Scroll	Scroll animado entre secções da página.
Optimistic Update	Atualizar a UI antes da resposta do servidor (rollback em caso de erro). Sensação de rapidez.
OnPush Change Detection	Estratégia Angular onde o component só atualiza quando recebe novos inputs. Melhora performance.

Backend (ASP.NET)

Termo	Significado
Controller	Classe C# que define endpoints REST. Recebe pedidos HTTP e devolve respostas.
Action	Método público de um Controller. Cada action = 1 endpoint.
DTO (Data Transfer Object)	Classe com apenas dados, usada para enviar/receber via API. Nunca expor entidades de BD diretamente.
Entity / Modelo	Classe C# que mapeia para uma tabela. Ex: <code>User</code> , <code>Post</code> , <code>Comment</code> .
DbContext	Classe do Entity Framework que representa a sessão com a BD (<code>Users</code> , <code>Posts</code> , etc. são <code>DbSet<></code>).
Migration	Snapshot versionado de mudanças à BD. <code>Add-Migration InitialCreate</code> gera o ficheiro; <code>Update-Database</code> aplica.
EF Core (Entity Framework)	ORM (Object-Relational Mapper). Traduz código C# em SQL. Permite escrever LINQ em vez de SQL puro.
LINQ	Language Integrated Query. Sintaxe C# para queries (<code>Where</code> , <code>Select</code> , <code>OrderBy</code> ...).
Code-First	Abordagem onde escreves as classes C# e o EF gera as tabelas. (Alternativa: Database-First.)
Repository	Camada que isola o acesso a dados. Esconde detalhes do EF Core.
Service	Camada que contém a lógica de negócio (regras). Chama repositórios.
Dependency Injection (DI)	Mecanismo .NET que entrega dependências às classes (em vez de criar com <code>new</code>).
Middleware	Componente do pipeline ASP.NET. Cada pedido passa por uma cadeia de middlewares (auth, CORS, exceptions, controllers).
<code>[Authorize]</code>	Atributo que protege um endpoint — só utilizadores autenticados acedem.
Claims	Pares chave-valor dentro do JWT que identificam o utilizador (<code>sub</code> , <code>email</code> , <code>role</code>).
BCrypt	Algoritmo de hashing de passwords. Nunca guardar passwords em claro.
AutoMapper	Biblioteca que mapeia automaticamente Entidade ↔ DTO.
FluentValidation	Biblioteca para validar DTOs com regras encadeadas.

Termo	Significado
Swagger / OpenAPI	Documentação interativa da API. Vê todos os endpoints e testas no browser (<code>/swagger</code>).
<code>IFormFile</code>	Tipo .NET que representa um ficheiro enviado via multipart.
<code>async</code> / <code>await</code>	Sintaxe C# para código assíncrono não-bloqueante. Toda a API deve ser <code>async</code> .

Base de Dados (SQL Server)

Termo	Significado
Tabela	Conjunto de linhas com a mesma estrutura. Ex: <code>Users</code> .
PK (Primary Key)	Coluna(s) que identificam unicamente cada linha. Ex: <code>Id</code> .
FK (Foreign Key)	Coluna que aponta para a PK de outra tabela (ex: <code>Posts.UserId</code> → <code>Users.Id</code>).
UNIQUE constraint	Garante que valores duma coluna (ou combinação) não se repetem. Ex: <code>(PostId, UserId)</code> em <code>Bazes</code> .
CHECK constraint	Validação a nível de BD. Ex: <code>FollowerId <> FollowedId</code> .
Índice (Index)	Estrutura que acelera queries em colunas. Ex: <code>IX_Posts_CreatedAt</code> .
3FN (Terceira Forma Normal)	Critério de normalização. Sem dados duplicados, dependências apenas da PK.
JOIN	Operação SQL que combina linhas de duas tabelas.
<code>GETUTCDATE()</code>	Função SQL Server que devolve a data/hora UTC atual.
SSMS	SQL Server Management Studio. Ferramenta gráfica para inspecionar a BD.

Git / Colaboração

Termo	Significado
Repositório (repo)	Pasta versionada com Git.
Branch	"Ramo" paralelo de desenvolvimento. Ex: <code>develop</code> , <code>feature/auth</code> .
Commit	Snapshot guardado no histórico.
Push / Pull	Enviar / receber commits do/para o remote (GitHub).
Merge	Juntar uma branch a outra.
Pull Request (PR)	Pedido formal para juntar uma branch (<code>feature/auth</code>) à <code>develop</code> , com revisão.
Conflito	Quando dois commits alteram a mesma linha de um ficheiro. Resolve-se manualmente.
Tag	Marca um commit como versão. Ex: <code>v0.5-parcelar</code> .
Monorepo	Um único repositório para backend + frontend (ao contrário de 2 repos separados).

Estrutura de domínio NzolaNet

Termo	Significado
Baze	Reação do utilizador a uma publicação (equivalente cultural angolano do "like"). Aceita-se manter este nome no código por ser termo do domínio.
Follow	Relação onde um utilizador segue outro.
Feed	Lista cronológica de publicações.
Notification	Aviso gerado automaticamente (novo baze, comentário, seguidor).

Nota técnica sobre inglês: todo o código (classes, métodos, tabelas, ficheiros) está em inglês porque é convenção universal — frameworks como Entity Framework, ASP.NET e Angular esperam nomes em inglês, a documentação oficial e Stack Overflow estão em inglês, e termos como `Repository` , `Service` , `Controller` , `Middleware` , `Token` não traduzem bem. A exceção é `Baze` (e potencialmente outros termos do domínio NzolaNet) que são culturalmente específicos. **Comentários e mensagens ao utilizador final** ficam em português.

Foco. Disciplina. Entrega. May the code be with you.